

CSCE 611 OPERATING SYSTEMS · MODULE 04 · LECTURE NOTES

Synchronization

Making concurrent access to shared state correct.

MODULE OUTCOMES

- Explain race conditions and critical sections.
- Implement a spinlock and explain what hardware support it needs.
- Explain why spinning is sometimes right and usually wrong.
- Use condition variables correctly, including why the wait is a loop.
- Choose between locks, lock-free structures, and message passing.

1 Race conditions

```
counter++;

// compiles to three separate instructions:
ld  t0, counter    // read
addi t0, t0, 1     // modify
sd  t0, counter    // write
```

Two threads executing this concurrently can interleave so that both read the same value, both increment it, and both write back the same result. Two increments produce one. The outcome depends on timing that neither thread controls.

This is a **race condition**, and its defining practical property is that it is intermittent. The bad interleaving may occur once in a million executions, which means it passes every test and appears in production under load.

1.1 What a solution must provide

Property	Requirement
Safety (mutual exclusion)	Never more than one thread inside the critical section.
Liveness (progress)	If no thread is in the critical section and some are waiting, one of them must eventually enter. No deadlock.
Bounded waiting	A waiting thread is not overtaken indefinitely. No starvation.

All three matter. A lock that is safe but can deadlock is not usable; one that is safe and live but permits a thread to wait forever while others repeatedly acquire is not fair.

2 Building a lock

WHY HARDWARE SUPPORT IS NECESSARY

The obvious lock — read the flag, and if it is clear, set it — has exactly the problem it is meant to solve. Two threads can both read 'clear' before either writes.

Pure software solutions exist. Peterson's algorithm achieves mutual exclusion for two threads with only loads and stores. It does not generalise cleanly beyond two, and critically it depends on sequentially consistent memory, which no real processor provides (CSCE 614 Module 12). On actual hardware it requires explicit barriers to work at all.

So processors provide an **atomic read-modify-write** instruction: test-and-set, compare-and-swap, or fetch-and-add. The hardware obtains the cache line in Modified state (CSCE 614 Module 11) and completes the entire operation before releasing it.

Every lock on every platform is built on one of these primitives.

```
void acquire(struct spinlock *lk) {
    push_off();                // disable interrupts, this CPU
    while (__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;                      // spin while it was already 1
    __sync_synchronize();      // barrier
    lk->cpu = mycpu();
}

void release(struct spinlock *lk) {
    lk->cpu = 0;
    __sync_synchronize();      // barrier
    __sync_lock_release(&lk->locked);
    pop_off();                 // restore interrupts
}
```

- **The barriers are mandatory.** Without them, the compiler or the processor may move loads and stores from inside the critical section to outside it. The lock would be acquired correctly and protect nothing. CSCE 614 Module 12 covers why.
- **push_off disables interrupts.** If a thread holds a spinlock and an interrupt handler on the same CPU tries to take it, the handler spins forever waiting for a thread that cannot resume until the handler returns. Disabling interrupts while holding the lock prevents this self-deadlock.

NEVER SLEEP WHILE HOLDING A SPINLOCK

A spinning thread occupies a CPU doing no work. That is a reasonable trade only when the holder will release within a few hundred cycles — less than the cost of two context switches.

If the holder blocks on I/O while holding the lock, every spinner burns an entire CPU for the duration — possibly milliseconds.

On a uniprocessor it is worse than wasteful: the spinner never yields, so the holder is never scheduled, so the lock is never released. The system stops entirely.

Spinlocks protect short, non-blocking critical sections. Anything that could sleep needs a sleeping lock.

	Spinlock	Sleeping lock (mutex)
While waiting	Busy-waits, occupying a CPU.	Blocks; the scheduler runs something else.
Latency to acquire	Very low — no context switch.	Two context switches.
Cost of waiting	A whole CPU.	Nothing.
Use when	The critical section is a handful of instructions, you are on multicore, and the code cannot block.	The critical section may be long, or may block on I/O, or you are in user space.
Typical use	Kernel data structures, run queues, free lists.	Essentially everything else.

3 Condition variables

A lock provides mutual exclusion. It does not provide a way to wait for a *condition* — for the queue to become non-empty, for a child to exit, for a buffer to have space. Polling in a loop while holding the lock would prevent anyone from ever making the condition true.

```
// CONSUMER
acquire(&lock);
while (queue_is_empty())           // WHILE, not IF
    sleep(&queue, &lock);          // atomically release lock + block;
                                   // reacquire lock before returning

item = dequeue();
release(&lock);

// PRODUCER
acquire(&lock);
enqueue(item);
wakeup(&queue);
release(&lock);
```

SLEEP MUST RELEASE AND BLOCK ATOMICALLY

Suppose sleep released the lock and then blocked as two separate steps. A producer could run in the gap: it acquires the lock, enqueues an item, calls wakeup — and nobody is asleep yet, so the wakeup is discarded.

The consumer then blocks, waiting for an event that has already happened. It sleeps forever with an item sitting in the queue.

This is the **lost wakeup** problem, and it is why sleep takes the lock as a parameter: it holds a separate internal lock, marks the thread as sleeping, releases the caller's lock, and only then yields — so no wakeup can slip through the gap.

3.1 Why the wait is a while loop

THREE INDEPENDENT REASONS, EACH SUFFICIENT ON ITS OWN

Spurious wakeups. Implementations are permitted to wake a thread with no corresponding signal, and some do — it simplifies the implementation and is explicitly allowed by POSIX and by the C++ standard.

Multiple waiters, one resource. A wakeup may wake every waiter. One of them gets the item; the others find the queue empty again and must sleep.

Stolen condition. Between being woken and successfully reacquiring the lock, a different thread may have entered and consumed the item.

In every case the woken thread must **re-test the condition**. Writing if instead of while produces a bug that is rare, timing-dependent, and extremely difficult to reproduce — which is why it is worth learning as a reflex rather than as a judgement call.

4 The toolkit

Primitive	Provides	Hazards
Spinlock	Mutual exclusion by busy-waiting.	Never sleep while holding one. Always disable interrupts if an interrupt handler may take the same lock.
Mutex	Mutual exclusion by blocking.	Deadlock (Module 05). Must be released by the thread that acquired it.
Condition variable	Waiting for a state change, with a lock.	Always use while. Signal while holding the lock to avoid races.
Semaphore	A counter with atomic wait and post.	No concept of an owner, so misuse is easy and priority inheritance is impossible. Good for counting resources; poor as a general mutex.
Reader-writer lock	Concurrent readers, exclusive writers.	Writer starvation under continuous read load. Often slower than a plain mutex unless reads are both frequent and long.
RCU	Read-mostly structures with <i>zero-cost</i> readers.	Readers take no lock at all; writers publish a new version and defer reclaiming the old one until all pre-existing readers have finished. Powerful, and complex. Heavily used in Linux.

5 Practical advice

- 1 Reduce sharing before synchronising it.** Per-thread state combined once at the end eliminates the problem rather than making it safe. CSCE 614 Module 11 showed this is also faster.
- 2 Use a plain mutex unless you have measured a need for something else.** Uncontended locks cost about twenty cycles; nearly all lock cost is contention cost.
- 3 Establish and document a lock ordering.** This is the main defence against deadlock, and Module 05 develops it.
- 4 Keep critical sections short,** and never perform I/O inside one if it can be avoided.
- 5 Do not invent synchronisation primitives** without a written correctness argument. This is the area where plausible reasoning is least reliable.
- 6 Run everything under ThreadSanitizer.** It detects races that no amount of testing will surface, because it reasons about happens-before rather than about observed outcomes.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
OSTEP — Chapters 26–31 (Concurrency, Locks, Condition Variables, Semaphores)	The clearest written treatment of locks and condition variables, including the lost-wakeup derivation.
xv6 book — Chapter 6 (Locking)	Real spinlocks, including the interrupt and memory-barrier reasoning.
Paul McKenney — Is Parallel Programming Hard? (free book)	The practitioner's reference, and the definitive treatment of RCU by its author.
Jeff Preshing — locks and lock-free programming	Excellent on memory ordering in lock implementations. Pairs with CSCE 614 Module 12.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write a program where two threads increment a shared counter ten million times without synchronisation. Report the final value across twenty runs and observe the variation.
- 2 Implement a spinlock with an atomic test-and-set. Verify it fixes the counter, then remove the memory barriers and look for reordering effects in the generated assembly.
- 3 Deliberately sleep while holding a spinlock in xv6 on a single CPU and observe the deadlock. Explain precisely why it cannot recover.
- 4 Implement a bounded producer-consumer queue with a mutex and two condition variables. Test with multiple producers and consumers.
- 5 Change the condition variable wait from while to if and construct a workload that exposes the bug. This may take effort, which is itself the lesson.
- 6 Implement the lost-wakeup bug by separating release and sleep into two steps, and demonstrate a consumer blocking forever with an item in the queue.
- 7 Measure uncontended and contended mutex cost against spinlock cost at 1, 2, 4, and 8 threads, with a short and a long critical section. Identify where each wins.
- 8 Run any of the above under ThreadSanitizer and compare what it reports against what your tests found.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is counter++ not atomic, and what makes the resulting bug so hard to catch?

- 2 Name the three properties a mutual exclusion solution must provide.
- 3 Why can a lock not be built from ordinary loads and stores on real hardware?
- 4 Give two reasons a spinlock implementation needs memory barriers, and one reason it disables interrupts.
- 5 What happens if you sleep while holding a spinlock on a uniprocessor?
- 6 Why must sleep release the lock and block atomically?
- 7 Give three independent reasons a condition variable wait must be a while loop.